

C++ Programming: Systems And Programming Concepts

Chapter 1

Advanced Arrays Manipulation

Array - One dimensional

Is a collection of a fixed number of elements of the same data type.

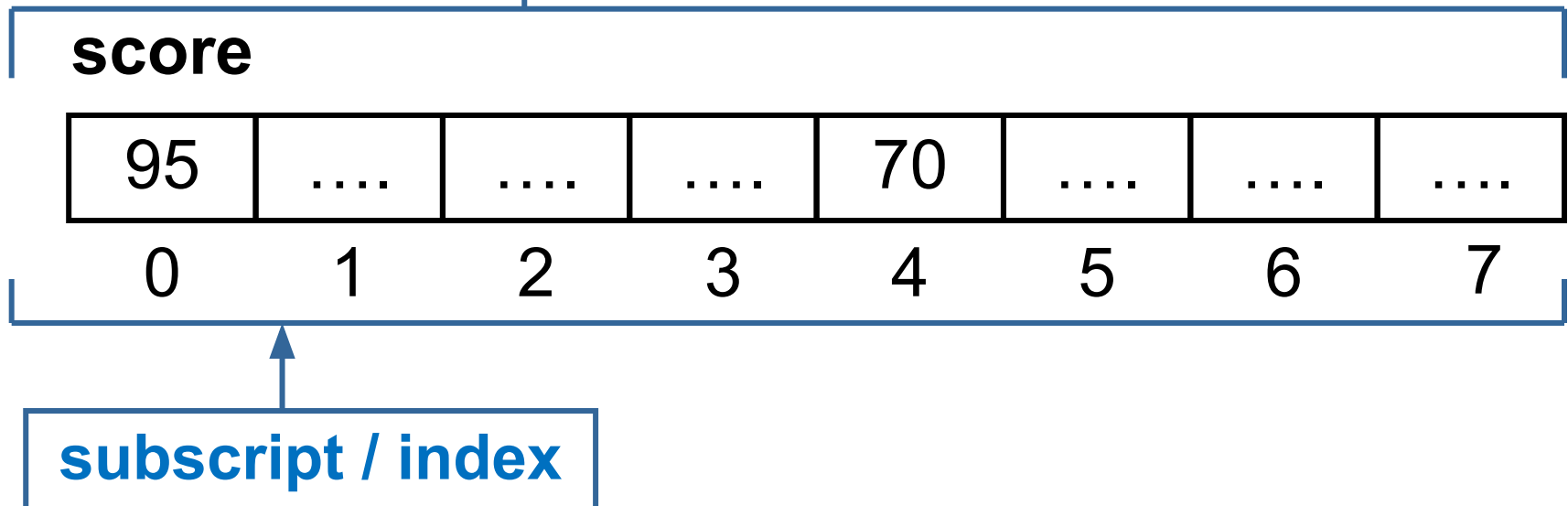
One-dimensional array - an array in which the elements are arranged in a list form.

Each element is accessed by using **index / subscript**. It always starts from **zero**.

Array - Example

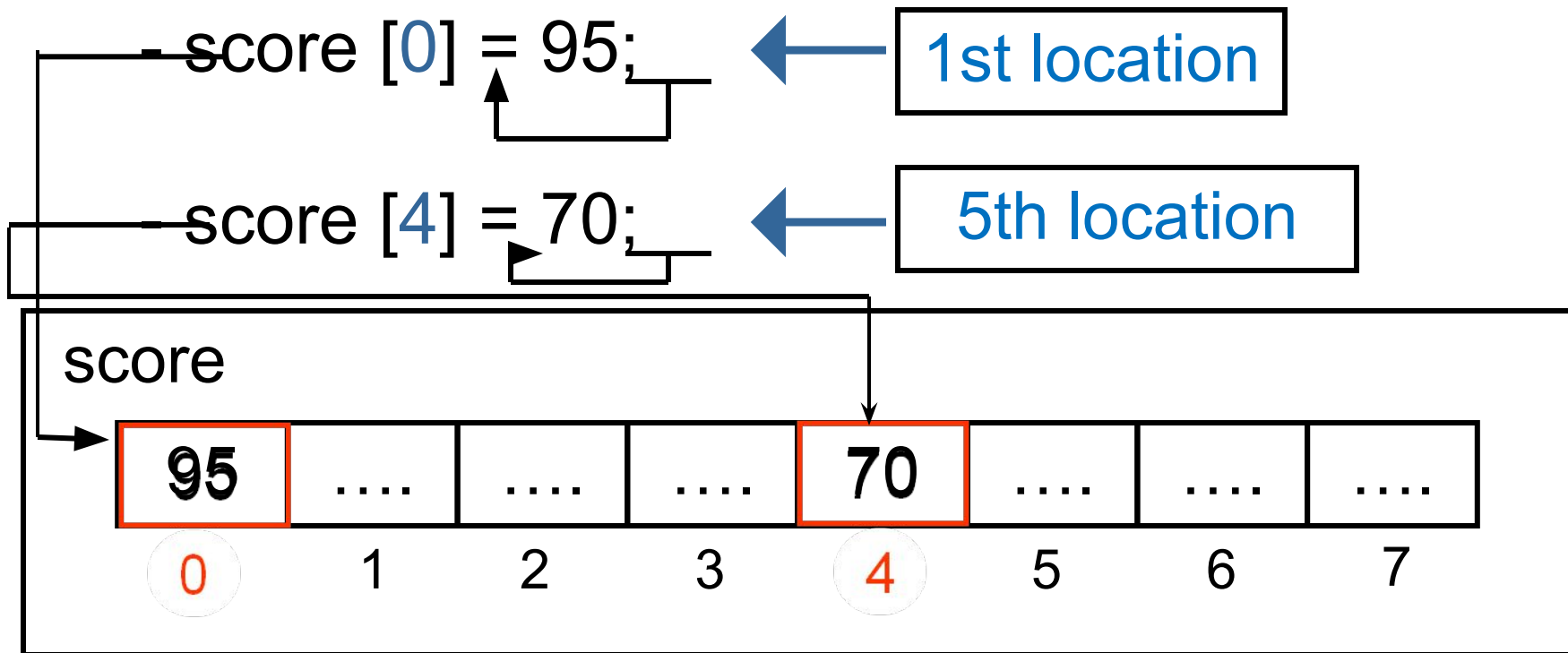
Arrays allow us to store related data items using the same identifier.

Example: An array named **score** which can store 8 different values at the same time.



Array - Example

Example: To store the values **95** and **70** into **first** and **fifth** locations respectively:



Array – Declaration / Definition

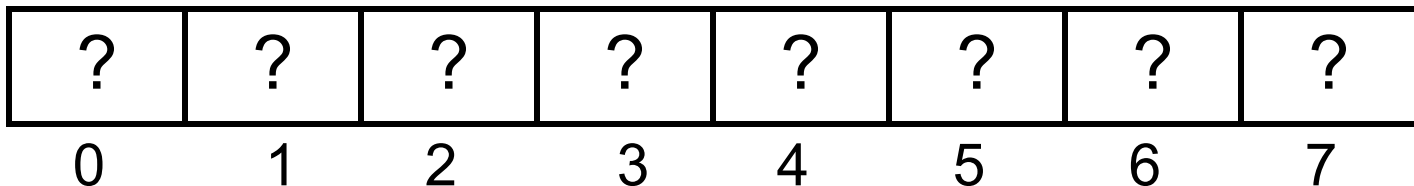
Array declaration:

<data type> <array name> [<array size>]

E.g.

int score [8];

Defines an array which can store **8** integers



Array components-Assignment

Consider the array

```
int score[8];
```

Its elements are:

```
score[0], score[1], ....., score[7]
```

Using assignment statements

```
score[3] = 88; // assign 88 to 4th element
```

```
i = 2; score[i] = 45; // assign 45 to 3rd  
element
```

```
score[7] = score[7] + 5; // add 5 to the last  
// element of score array
```

Array – Initialization

Array initialization:

Initialization with definition:

compile-time initializing

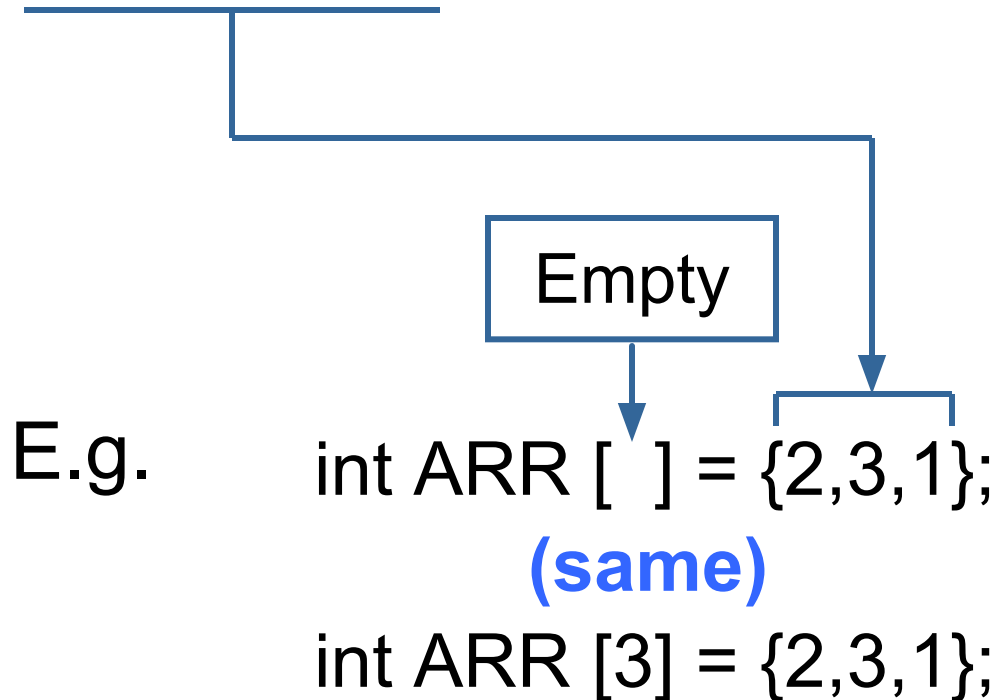
Braces { } are used to enclose the elements, which must be separated by commas (,).

E.g. `int score [8] = { 95, 08, 85, 67, 70, 60, 78, 90 };`

`float costs [4] = { 12.00, 5.00, 2.50, 7.25 };`

Array Declaration & Initialization

The **squares brackets []** can be **left empty** if **all elements** are present in the brackets.



Operation on Array

Initialize directly using a program statement:

```
int testscores[10];  
testscores[5] = 60;
```

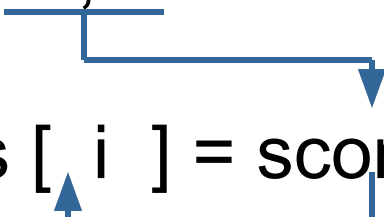
Compile-time initialization using a program statement:

```
int testscores[10] = {60, 20, 11, 55, 45, 89, 75 ,85, 23, 45};
```

Processing of the whole array requires the use of **for** loop

Operation on Array

```
int testscores[8];  
for (i= 0 ; i<8; i++){  
    cin >> score;  
    testscores [ i ] = score;  
}
```



Run-time

67
50
95
.....

testscores

67	50	95					
0	1	2	3	4	5	6	7

Operation on Array - Example

Assume that we have the following declarations:

```
double sales[10];
```

```
int index;
```

```
double largest, sum=0, average;
```

Initializing every elements of the array **sales** to **0.0**

```
for(index = 0; index < 10; index++)
```

```
    sales[index] = 0.0;
```

Operation on Array - Example

Reading data into the array sales

```
for(index = 0; index < 10; index++)  
    cin >> sales[index];
```

Printing the values from the array sales

```
for(index = 0; index < 10; index++)  
    cout << sales[index];
```

Operation on Array - Example

- Find the **sum** and **average** from the array **sales**

```
int sum = 0;
```

```
for(index = 0; index < 10; index++)
```

```
    sum += sales[index];
```

```
average = sum / 10.0;
```

- Find the **largest** value from the array **sales**

```
largest = sales[0];
```

```
for(index = 1; index < 10; index++)
```

```
    if (largest < sales[index])
```

```
        largest = sales[index];
```

Manipulation of Array elements

To double the value of all elements in testscore:

Before `int testscore[ 8 ] = { 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 };`
`for ( i=0; i< 8; i++)`
`testscore[i] *= 2;` **After**

testscore

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

After:

testscore

2	4	6	8	10	12	14	16
---	---	---	---	----	----	----	----

Manipulation of Array elements

To print the contents of an array ARR:

```
int ARR[4] = {5, 7, 9, 11};  
for ( i=0; i<4; i++)  
    cout<< "ARR[" << i << "] ="<<ARR[i] << endl;
```

```
Output:  ARR[0] = 5  
        ARR[1] = 7  
        ARR[2] = 9  
        ARR[3] = 11
```

Manipulation of Array elements

From one array, ARR, produce another array, SQUARE:

```
int ARR[8] = { 2, 1, 9, 6, 5, 4, 7, 3 };  
int SQUARE[8];  
:  
for ( i= 0; i< 8; i++)  
    SQUARE[i] = ARR[i] * ARR[i];
```



SQUARE

4	1	81	36	25	16	49	9
---	---	----	----	----	----	----	---

Manipulation of Indexes

- The index can be written in many ways.

- Example:

```
int x[5]={1, 2, 3, 8, 7};  
int i = 2;
```

$x[i+1]$ refers to the 4th element, $x[3]$

$x[2*i]$ refers to the 5th element, $x[4]$

$x[0]$ has a value of 1

Array – Two dimensional

E.g ARR_2D is a 2-D array with 3 x 4 elements

0	..	..	..	..
1	..	..	..	7
2	..	..	..	..
	0	1	2	3

e.g ARR_2D[1][3]=7;

The element is accessed by using:

array_name[row number][column number]

Array – Two dimensional

Contents are listed **row by row**, or each row contents are enclosed separately in braces.

ARR_2D	0	2	4	6	8
	1	1	3	5	7
	2	1	4	9	16
		0	1	2	3

e.g `int ARR_2D[3][4]={ { 2,4,6,8 },
 { 1,3,5,7 },
 { 1,4,9,16 } };`

Array – Two dimensional

Only differences between single and two-dimensional arrays : **declaration** and **referencing**.
This is due to the *second index/dimension*

	One-Dimensional	Two-Dimensional
Declaration	<code>int Arr[5];</code>	<code>int Arr[2][3];</code>
Referencing / Initialization	<code>int Arr[5] = {1, 2, 3, 4, 5};</code> <code>cout << Arr[2];</code>	<code>int Arr[2][3]</code> <code>= {{2, 2, 2}, {3, 3, 3}};</code> <code>cout << Arr[1][2];</code>

Two dimensional array – Processing Algorithm

ARR_2D

0	2	4	6	8
1	1	3	5	7
2	1	4	9	16
	0	1	2	3

Single Row processing:

E.g. Sum up total of 2nd row of ARR_2D array

row = 1;

for (col = 0; col < 4; col++)

sum += ARR_2D [row][col];

Two dimensional array – Processing Algorithm

ARR_2D

0	2	4	6	8
1	1	3	5	7
2	1	4	9	16
	0	1	2	3

Single Column processing:

E.g. Sum up total of 4th column of ARR_2D array

col = 3;

for (row = 0; row < 3; row++)

sum += ARR_2D [row][col];

Two dimensional array – Processing Algorithm

ARR_2D

0	2	4	6	8
1	1	3	5	7
2	1	4	9	16
	0	1	2	3

Row by Column processing:

E.g. Find the grand total of ARR_2D

```
for ( row = 0; row < 3; row++ )  
for ( col = 0; col < 4; col++ )  
    GrandTotal += ARR_2D [ row ][ col ];
```

Two dimensional array – Processing Algorithm

ARR_2D

0	2	4	6	8
1	1	3	5	7
2	1	4	9	16
	0	1	2	3

Column by Row processing:

E.g. Find the grand total of ARR_2D

```
for ( col = 0; col < 4; col++ )  
for ( row = 0; row < 3; row++ )  
    GrandTotal += ARR_2D [ row ][ col ];
```

Two dimensional array - Exercise

- a) Declare an integer type 2D array called *hotel* for a hotel which has 25 levels, each level has 15 rooms. The array records the number of guests who will stay in the room.

(Note: Room number 23-9 represents room at level 23, room unit 9)

- b) A travel agency has reserved the hotel rooms for the tourists which will occupy the whole level 11 and 12. Prompt user to enter the number of guests to stay in each room of these 2 levels.

Two dimensional array - Exercise

- c) You are asked to make a correction for the number of guests staying in the Room number 18-5 to 4.
- d) The hotel manager likes to know the total of guests staying at the 2 levels reserved by the travel agency.

Vector Array

- Vector is a dynamic array implemented as a class. Methods/functions of the class can be accessed for vector.
- The array size is not fixed but can grow and shrink during run time where elements can be added or removed
- To include <vector> header file
- Vector can be declared of a data type

Syntax: **vector <data type> vectorName;**

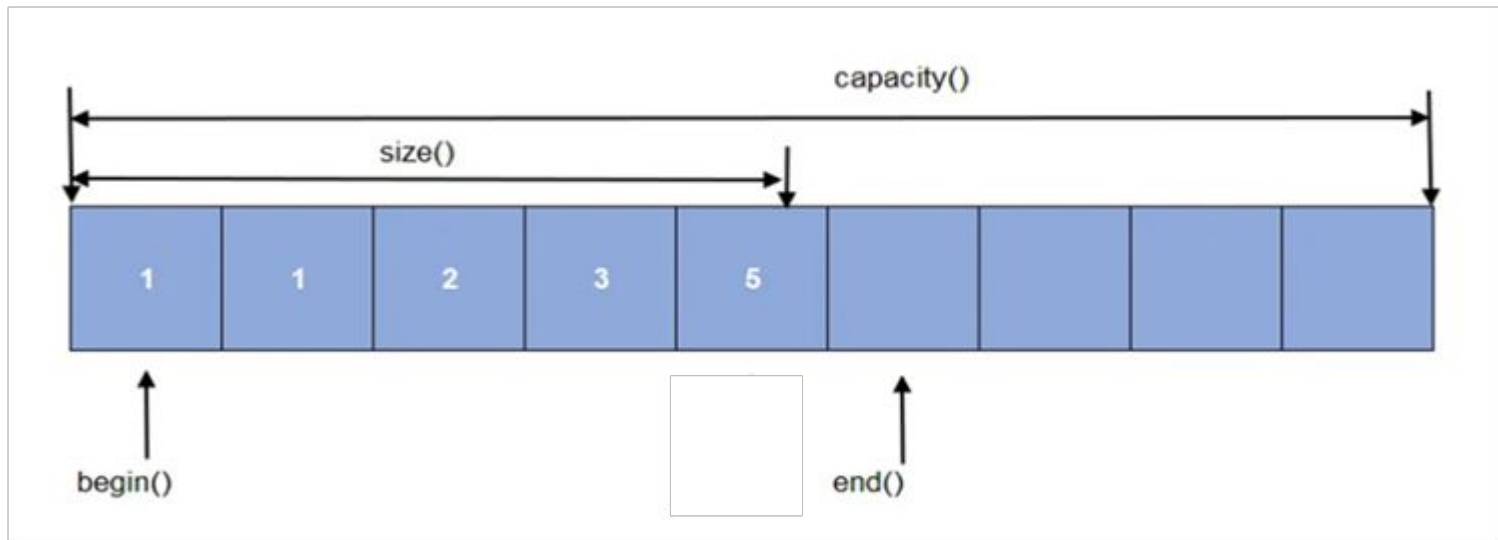
Vector Array – basic functions

Function	Description	Example
<code>push_back(val)</code>	Adds val to the end of the vector	<code>vec.push_back(5);</code>
<code>pop_back()</code>	Removes the last element	<code>vec.pop_back();</code>
<code>size()</code>	Returns number of elements	<code>vec.size();</code>
<code>empty()</code>	Returns true if vector is empty	<code>vec.empty();</code>
<code>clear()</code>	Removes all elements	<code>vec.clear();</code>
<code>at(index)</code>	Access element at a specific index (safe)	<code>vec.at(1);</code>

Vector Array – basic functions

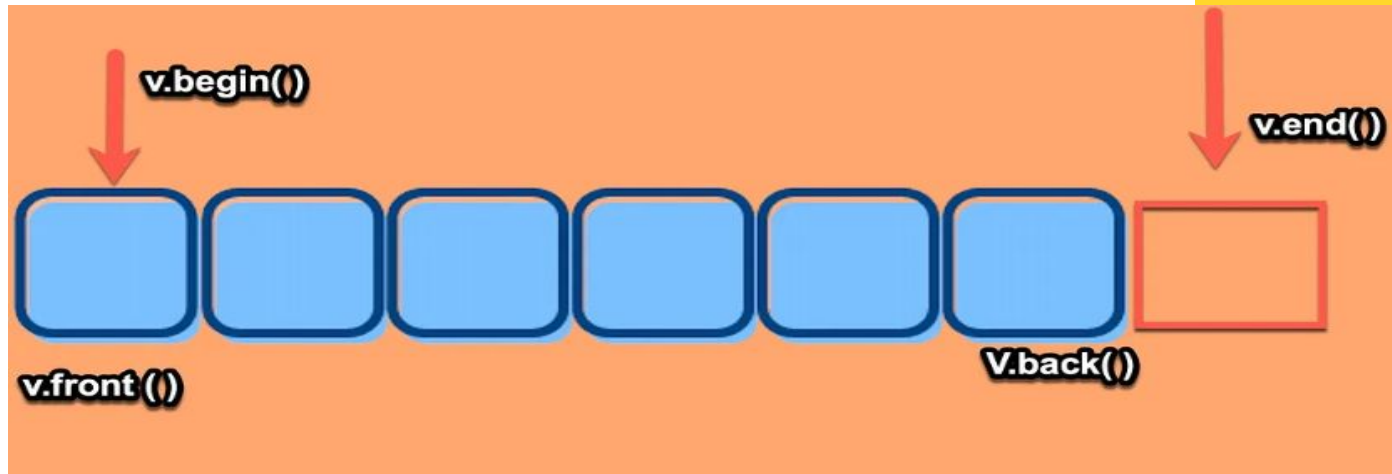
Function	Description	Example
[]	Access element (no bounds check)	<code>vec[0];</code>
<code>front()</code>	Returns the first element	<code>vec.front();</code>
<code>back()</code>	Returns the last element	<code>vec.back();</code>
<code>insert(pos, val)</code>	Inserts val at pos	<code>vec.insert(vec.begin() + 2, 10);</code>
<code>erase(pos)</code>	Removes element at pos	<code>vec.erase(vec.begin() + 1);</code>
<code>resize(n)</code>	Changes the size of the vector to n	<code>vec.resize(5);</code>
<code>swap(other)</code>	Swaps content with another vector	<code>vec1.swap(vec2);</code>

Vector Array - a container



- The `size()` indicates the number of elements stored in the vector currently
- The `capacity()` indicates the maximum elements it can hold before having to relocate additional memory

Vector Array - a container



Note:

- `begin()` and `front()` are not the same
- `begin()` refers to pointer/iterator points to 1st element, useful in looping thru the vector
- `front()` is used to access to the value of the 1st element of the vector array

Vector Array - a container

- `end()` and `back()` work similar like the `begin()` and `front()`
- `begin()` refers to pointer/iterator points to an element, useful in looping thru the vector
- `front()` is used to access to the value of the first element of the vector
- Iterator (pointer) provides a way to traverse and manipulate the elements in the vector
- `*` is a dereferencing operator used to access and replace the element's value where the iterator is pointing to.

Vector Array - Example

```
#include <iostream>
#include <vector>
using namespace std;
void main() {
    // Declare vector array num
    vector<int> num = { 1,2,3,4,5 };

    // Add 10 and 20 at the end of
    // num array
    num.push_back(10);
    num.push_back(20);

    // Display all the numbers in
    // the array.
    // Output: 1,2,3,4,5,10,20
    for(int i = 0; i < num.size(); i++)
        cout << num[i] << " ";

    // Remove last element from array
    num.pop_back();

    // Remove third element from array
    num.erase(num.begin()+2);

    // Insert 30 to second element
    num.insert(num.begin()+1, 30);

    // Display all the numbers in the array
    // Output: 1,30,2,4,5,10
    for(int i = 0; i < num.size(); i++)
        cout << num[i] << " ";

    // Display first plus last element value
    // Output is 11
    cout << num.front() + num.back();
}
```

Vector Array - Example

```
// Declare an iterator
    vector<int>::iterator it = num.begin()+1;
// Display current value where the iterator is at
// Use * to access to the element's value
// Output is 30
    cout << "Current iterator value is ";
    cout << *it << endl;

// Move the iterator right 3 times
    it = it + 3;

// Replace the current value where the iterator
// is pointing to by 999
    *it = 999;

// Display all the numbers in the array
// Output: 1,30,2,4,999,10
for(int i = 0; i < num.size(); i++)
    cout << num[i] << " "; }
```